Trabajo Práctico 1

SISTEMAS OPERATIVOS

Índice

1. Iniciando en Linux **3**

2. Línea de comandos **3**

3. Carpetas y archivos **3**

4. Descriptores de archivos **4**

5. Comandos **4**
5.1. Algunos comandos 4
5.2. APT: administrador de paquetes 7

6. Llamadas a sistema, "syscalls" **8**

7. Editores de texto **9**
7.1. Mini-Tutorial de VIM 9

8. C: hello, world **9**
8.1. GCC: GNU Compiler Collection 10

9. Python: hello, world **12**

10. Permisos de archivos **12**

11. Time **13**

1. Iniciando en Linux

Para iniciar este viaje, te invitamos a encender la computadora y elegir bootear con "Ubuntu", el Sistema Operativo que utilizaremos en la materia.

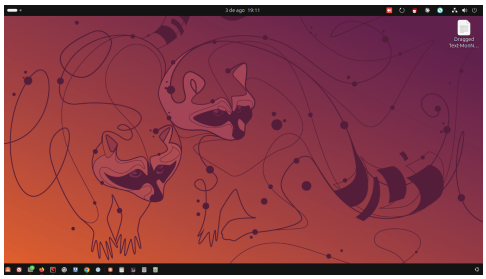


Figura 1: Escritorio de Ubuntu

2. Línea de comandos

La gran mayoría de las distribuciones de Linux proveen acceso a una interfaz basada en **texto**.

En general, escribimos un comando, apretamos `Enter`, se ejecuta y muestra el resultado.

Podemos abrir una terminal abriendo el menú de aplicaciones y seleccionando “Terminal”, o simplemente apretando `Ctrl+Alt+T`.

Esto abre `gnome-terminal` que es un emulador de terminal.

Dentro vamos a ver una linea similar a:

```
ubuntu@ubuntu:~$
```

Esta se llama “prompt” y singifica que esta a la espera de un comando.

Se puede intepretar de la siguiente manera:

`ubuntu` ← nombre de usuario
`ubuntu#` ← se lee “at” y significa “en”
`ubuntu@ubuntu` ← el nombre de la maquina
`ubuntu@ubuntu:` ← separador
`ubuntu@ubuntu:~` ← Carpeta actual
`ubuntu@ubuntu:~$` ← Nos indica que estamos en modo usuario; `#` significa `root`.

3. Carpetas y archivos

Los archivos dentro del sistema de archivos de linux tienen un “path” o “ruta” en el que estan ubicados.

Estos “paths” tienen unas nomenclaturas especiales:

Si el "path" empieza con "/" entonces es un path "absoluto".
Si el "path" empieza con un nombre entonces es un path "relativo" a la carpeta actual.
El caracter "~" es especial y significa el "home directory" o "directorio casa" para el usuario actual.
El caracter "." es especial y es una referencia a la carpeta actual.
El conjunto ".." es especial y significa la "carpeta padre de".

4. Descriptores de archivos

Un descriptor de archivo "file descriptor" es un número entero no negativo que utiliza el sistema operativo para identificar de forma única un archivo abierto (entre otras cosas).

Por defecto, casi todos los programas se inician automáticamente con tres canales de comunicación asignados a valores enteros fijos:

- 0 (`stdin`): Entrada estándar
- 1 (`stdout`): Salida estándar
- 2 (`stderr`): Error estándar

Si abrimos un archivo usando la llamada al sistema `open`, es posible que se nos asigne el descriptor "3".

5. Comandos

Los comandos que escribamos en la terminal son interpretados por un programa que se llama "shell". El "shell" mas comun es "bash".

La estructura basica de un comando en "bash" es `comando argumento1 argumento2 ...`

5.1. Algunos comandos

Un comando util es `pwd` - "Print working directory". Esto muestra como resultado la carpeta actual que tenemos abierta ("cwd" - Current Working Directory). Es posible ejecutar sin argumentos.

```
ubuntu@ubuntu:~$ pwd
/home/ubuntu
ubuntu@ubuntu:~$
```

Otro comando conveniente es `ls`. - "List directory contents". Lista las carpetas y archivos que se encuentan en el "cwd".

```
ubuntu@ubuntu:~$ ls
Desktop Documents Downloads Music Pictures Public Templates Videos
    snap
ubuntu@ubuntu:~$
```

Es posible cambiar de carpeta con `cd` - "Change directory".

```
ubuntu@ubuntu:~$ cd Pictures/
ubuntu@ubuntu:~/Pictures$ pwd
/home/ubuntu/Pictures
ubuntu@ubuntu:~/Pictures$ ls
Screenshots
ubuntu@ubuntu:~/Pictures$
```

Tambien es posible volver a la carpeta anterior con "`cd -`"

INFO

Tambien esta `pushd` y `popd` para navegar!

El comando `cat` - "Concatenate" permite concatenar archivos y mostrar su resultado. Es posible "concatenar" un solo archivo; y de esa forma, mostrarlo por pantalla.

```
ubuntu@ubuntu:~$ cat /proc/cpuinfo
processor : 0
vendor_id : AuthenticAMD
cpu family : 25
model : 33
model name : AMD Ryzen 9 5950X 16-Core Processor
stepping : 0
microcode : 0xa201030
cpu MHz : 4066.156
cache size : 512 KB
physical id : 0
siblings : 32
core id : 0
cpu cores : 16

[...]
```

Otro detalle interesante a notar es que acabo de mostrar un archivo del `procfs`.

INFO

En muchos sistemas operativos tipo UNIX se utiliza el sistema de archivos `proc` (proc filesystem o `procfs`), que es un sistema de archivos, generado dinámicamente, que muestra información de procesos. Es un sistema de archivos que no tiene relación con un dispositivo de almacenamiento.
Está implementado en el núcleo del sistema operativo y permite llevar a «espacio de usuario» datos del núcleo y de los procesos, que, sin la ayuda de `procfs`, sería casi imposible accederlos.
Este sistema de archivos está montado normalmente en el directorio `/`

`proc` y, dado que no contiene archivos reales, no consume espacio de almacenamiento.
Como es un sistema de archivos es posible navegarlo con `cd` y `ls`.

EXTRA

Explorar que otras cosas interesantes hay aparte de `/proc/meminfo` o `/proc/interrupts`.

Otro comando extramadamente util en especial cuando empezas a usar Linux es `man` - "Manual".

Este comando permite leer los manuales de los comandos. Por ejemplo:

```
ubuntu@ubuntu:/proc$ man pwd
```

```
PWD(1) General Commands Manual PWD(1)

NAME
       pwd - Display the full filename of the current working
           directory.

SYNOPSIS
       pwd [-L|--logical] [-P|--physical] [-h|--help] [-V|--version]

DESCRIPTION
       Display the full filename of the current working directory.

OPTIONS
       -L, --logical
              use PWD from environment, even if it contains symlinks

       -P, --physical
              avoid all symlinks

       -h, --help
              Print help

       -V, --version
              Print version

VERSION
       v(uutils coreutils) 0.8.0

                                    2026-04-16 PWD(1)
```

INFO

En la SYNOPSIS, argumentos entre corchetes, como `\[-L|--logical\]` significa que el argumento es opcional. La barra vertical o "pipe" `|` significa que es equivalente escribir `-L` o `--logical`.

INFO

Man es un "pager". Un tipo de programa que permite navegar/scrollear para arriba y abajo con las flechas ↑ y ↓.
Se puede salir apretando `q`.
Tambien tiene otras funcionalidades como apretar `/` entra en modo busqueda. Es posible escribir un texto y apretar `Enter`. Esto resalta todas las ocurrencias del texto. Se puede saltar entre anterior y posterior con `N` y `Shift+N`.

5.2. APT: administrador de paquetes

APT (*Advanced Package Tool*) permite instalar, actualizar y eliminar programas en distribuciones basadas en Debian, como Ubuntu.

La información sobre los paquetes disponibles se actualiza con:

```
ubuntu@ubuntu:~$ sudo apt update
```

Este comando descarga la información más reciente desde los repositorios configurados. No instala ni actualiza programas.

Para instalar un paquete usamos:

```
ubuntu@ubuntu:~$ sudo apt install tree
```

Para eliminarlo:

```
ubuntu@ubuntu:~$ sudo apt remove tree
```

Podemos buscar paquetes por nombre o descripción:

```
ubuntu@ubuntu:~$ apt search tree
```

La opción `sudo` ejecuta el comando con permisos administrativos, necesarios porque la instalación modifica directorios y archivos protegidos del sistema.

Una secuencia habitual para instalar software es:

```
ubuntu@ubuntu:~$ sudo apt update
ubuntu@ubuntu:~$ sudo apt install gcc
```

INFO

`apt update` actualiza el índice de paquetes. `apt install` instala un paquete usando ese índice. Son operaciones distintas.

6. Llamadas a sistema, "syscalls"

Podemos utilizar el programa `strace` (system call trace) que sigue la traza a las llamadas a sistema y las señales de un proceso, interceptándolas, registrándolas y mostrándolas por pantalla (stderr, standard error).

Cada línea mostrada contiene el nombre de la llamada a sistema, seguido por sus argumentos en paréntesis y sus valores retornados.

Por ejemplo, si ejecutáramos el comando

```
ubuntu@ubuntu:~$ cat /dev/null
ubuntu@ubuntu:~$
```

Debería mostrarnos el contenido de un archivo (de contenido nulo en este caso) pero que al seguirle el rastro a las llamadas a sistema que efectúa con el comando:

```
ubuntu@ubuntu:~$ strace cat /dev/null
execve("/usr/bin/cat", ["cat", "/dev/null"], 0x7ffe9d9161f8 /* 57
    vars */) = 0
[...]
statx(AT_FDCWD, "/dev/null", AT_STATX_SYNC_AS_STAT, STATX_ALL, {
    stx_mask=STATX_ALL|STATX_MNT_ID, stx_attributes=0, stx_mode=
    S_IFCHR|0666, stx_size=0, ...}) = 0
openat(AT_FDCWD, "/dev/null", O_RDONLY|O_CLOEXEC) = 3
[...]
read(3, "", 65536) = 0
close(3) = 0
[...]
exit_group(0) = ?
```

Nos muestra las sucesivas llamadas al sistema que va efectuando el proceso `cat`.

Podemos destacar entre sus llamadas al sistema la que carga el ejecutable en memoria `execve("/bin/cat"...` y la que efectivamente abre el archivo para ver su contenido `openat("/dev/null")`.

Podemos observar que el resultado de la syscall `openat` es "3". Esto significa que el sistema operativo abrio el archivo y le asigno el descriptor de archivos 3. Luego vemos como se utiliza este descriptor para interactuar con el archivo: se llama `read(3, ...` y `close(3)`.

INFO

Podemos probar también con un programa simple como `pwd` y observar que logra su objetivo primordial haciendo una llamada a la función `getcwd`.

7. Editores de texto

Un editor de texto es un programa como cualquier otro; pero que nos permite editar texto.

Algunos conocidos con interfaz grafica son:

- VSCode
- IntelliJ
- Zed
- Emacs
- gedit / gnome-text-editor

Pero hay editores que tambien se pueden usar desde la linea de comandos, los mas conocidos son:

- `visual` / `vim` (Vi IMproved) / `nvim` (Neo VIM)
- `nano`
- `emacs` (Si, tambien se puede ejecutar en la terminal)
- `ed` / `sed`

7.1. Mini-Tutorial de VIM

Primero abrir un archivo nuevo o existente:

```
ubuntu@ubuntu:~$ vim a.txt
```

Podemos mover el cursor con las flechas; o `k l j h`.

Antes de poder empezar a escribir; tenemos que entrar en modo de `-- INSERT --`.

Esto lo logramos apretando `append` o `insert`.

Podemos escribir normalmente un texto como “test”.

Luego, para guardar el archivo, hay que primero salir de modo `-- INSERT --` con `ESC` y entrar en modo comando con `:`.

Luego el comando es `wq` (“write” + “quit”).

VIM tambien posee modo busqueda con `/`.

INFO

Para mas informacion es posible correr el comando `vimtutor`.

8. C: hello, world

Un programa muy simple en C (Crear un archivo `hello.c` y escribir este contenido):

```c
#include <stdio.h>
int main() {
   printf("hello, world\n");
   return 0;
}
```

La línea `#include <stdio.h>` es una directiva para el preprocesador del compilador.

Esta directiva le indica al compilador que debe incluir el contenido del archivo de cabeceras llamado `stdio.h` (standard input and output) dentro del programa.

Este archivo contiene definiciones de funciones tales como `scanf()` y `printf()`. (Por ser parte de la libreria estandar, tambien funciona `man scanf`!)

Por lo tanto, si utilizamos en nuestro programa alguna de estas funciones y no solicitamos la inclusión del archivo `stdio.h`, el compilador no sabrá qué hacer con esa función no definida y dará error en la compilación.

La ejecución del programa C comienza con la función principal `main()`.

La función de biblioteca `printf()` envía texto formateado a la pantalla.

Y la instrucción `return 0` devuelve un cero -que, por convención, indica que no hubo error-, finalizando efectivamente el programa.

8.1. GCC: GNU Compiler Collection

La colección de compiladores GNU es un conjunto de compiladores creados por el proyecto GNU. Originalmente GCC (y el comando `gcc`) significaba GNU C Compiler (compilador GNU de C), porque sólo compilaba el lenguaje C, pero posteriormente se extendió para compilar C++, Fortran, Ada y otros.

Utilizaremos este comando para compilar el archivo fuente `hello.c` recientemente creado con el editor de nuestra preferencia. Simplemente, mediante el comando:

```
ubuntu@ubuntu:~$ gcc hello.c -o hello
ubuntu@ubuntu:~$ ./hello
hello, world
```

Se lo compila; la opción `-o` se utiliza para indicar que el archivo de salida (output) a generar debe llamarse como se le indica a continuación, caso contrario, el archivo de salida siempre se llamará `a.out`, y se lo ejecuta colocando por delante `./`.

Podemos trazar las llamadas a sistema que hace este sencillo programa, como sabemos, anteponiendo el comando «strace» al ejecutable:

```
ubuntu@ubuntu:~$ strace ./hello
execve("./hello", ["./hello"], 0x7ffc13506650 /* 57 vars */) = 0
brk(NULL) = 0x633a28904000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
    -1, 0) = 0x7c05cbbbd000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or
    directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=137421, ...}) = 0
mmap(NULL, 137421, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7c05cbb9b000
```

```
close(3) = 0
openat(AT_FDCWD, "/usr/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|
    O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0
    \250\2\0\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@
    \0\0\0\0\0\0\0"..., 840, 64) = 840
fstat(3, {st_mode=S_IFREG|0755, st_size=2186512, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@
    \0\0\0\0\0\0\0"..., 840, 64) = 840
mmap(NULL, 2231696, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0
    x7c05cb800000
mmap(0x7c05cb828000, 1671168, PROT_READ|PROT_EXEC, MAP_PRIVATE|
    MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7c05cb828000
mmap(0x7c05cb9c0000, 319488, PROT_READ, MAP_PRIVATE|MAP_FIXED|
    MAP_DENYWRITE, 3, 0x1c0000) = 0x7c05cb9c0000
mmap(0x7c05cba0e000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|
    MAP_FIXED|MAP_DENYWRITE, 3, 0x20d000) = 0x7c05cba0e000
mmap(0x7c05cba14000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|
    MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7c05cba14000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
    -1, 0) = 0x7c05cbb98000
arch_prctl(ARCH_SET_FS, 0x7c05cbb98740) = 0
set_tid_address(0x7c05cbb98d68) = 40700
set_robust_list(0x7c05cbb98a20, 24) = 0
rseq(0x7c05cbb98680, 0x21, 0, 0x53053053) = 0
mprotect(0x7c05cba0e000, 16384, PROT_READ) = 0
mprotect(0x6339fee16000, 4096, PROT_READ) = 0
mprotect(0x7c05cbc02000, 8192, PROT_READ) = 0
prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=
    RLIM64_INFINITY}) = 0
getrandom("\x77\x74\x79\x5a\xba\x8a\xa7\xb8", 8, GRND_NONBLOCK) = 8
munmap(0x7c05cbb9b000, 137421) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0x3), ...}) =
    0
brk(NULL) = 0x633a28904000
brk(0x633a28925000) = 0x633a28925000
write(1, "hello, world\n", 13hello, world
) = 13
exit_group(0) = ?
+++ exited with 0 +++
```

También podemos generar el código assembler de nuestro programa C indicando la opción `-S`, de esta manera:

```
ubuntu@ubuntu:~$ gcc -S hello.c
```

Este comando invoca al preprocesador (cpp) sobre el archivo `hello.c`, realiza una compilación inicial y luego se detiene antes de ejecutar el ensam-

blador. Genera el archivo `hello.s`.

9. Python: hello, world

El programa equivalente escrito en lenguaje Python sería simplemente:

```
#!/usr/bin/env python3
print("hello, world")
```

```
ubuntu@ubuntu:~$ vim hello.py
ubuntu@ubuntu:~$ ./hello.py
bash: ./hello.py: Permission denied
ubuntu@ubuntu:~$ chmod u+x hello.py
ubuntu@ubuntu:~$ ./hello.py
hello, world
ubuntu@ubuntu:~$
```

Notar que es necesario otorgarle permisos de ejecucion al archivo utilizando `chmod`.

10. Permisos de archivos

En Linux, cada archivo tiene permisos para tres grupos de usuarios:
- propietario (`user` o `u`),
- grupo propietario (`group` o `g`),
- resto de usuarios (`others` o `o`).

Cada grupo puede tener permisos de lectura, escritura y ejecución:
- `r`: leer el contenido,
- `w`: modificar el contenido,
- `x`: ejecutar el archivo.

Podemos observarlos usando:

```
ubuntu@ubuntu:~$ ls -l hello.py
-rwxr--r-- 1 ubuntu ubuntu 48 Aug 3 20:00 hello.py
```

Los primeros diez caracteres representan el tipo de archivo y sus permisos. El primer carácter indica el tipo; por ejemplo, `-` representa un archivo regular. Los siguientes nueve caracteres se agrupan en propietario, grupo y otros:

```
-rwx r-- r--
 u g o
```

También es posible expresar permisos usando números. Los valores son:
- lectura: 4,
- escritura: 2,
- ejecución: 1.

Por ejemplo, `chmod 755 hello.py` asigna permisos de lectura, escritura y ejecución al propietario, y permisos de lectura y ejecución al grupo y al resto:

```
ubuntu@ubuntu:~$ chmod 755 hello.py
ubuntu@ubuntu:~$ ls -l hello.py
-rwxr-xr-x 1 ubuntu ubuntu 48 Aug 3 20:00 hello.py
```

Alternativamente es posible ejecutarlo directamente usando python: `python3 hello.py`

11. Time

Es posible medir el tiempo de ejecución a los comandos anteponiéndoles el comando `time`.

```
ubuntu@ubuntu:~$ time ./hello
hello, world

real 0m0.001s
user 0m0.000s
sys 0m0.001s
```

EXTRA

Haga estas prácticas tanto con el ejecutable del código C como con el de Python.
Observe las diferencias.
Compare las dos ejecuciones